The Learner Must Re-Derive: Procedure-Level Generalization Certificates for Abstract Reasoning

Chimdi Walter

Abstract

Benchmark accuracy cannot distinguish reasoning from recall: a system that memorizes and one that generalizes may score identically. We present a program-induction system for ARC in which a task counts as solved only if the *entire learning procedure*, re-run from $N-1$ of its training examples, re-derives a program that solves the held-out example—for every fold. This leave-one-out-by-reinduction gate produces a machine-checkable **procedure-level certificate** per solve. We show the certificate is measurably load-bearing: certified programs are correct on hidden tests 95.3% of the time versus 33.2% for the same search's uncertified train-perfect programs (a 2.9× truth gap that raw accuracy inverts—disabling the gate *raises* claimed solves 5.3× while collapsing their reliability). We further show that a syntactic preference lattice over parameter expressions (relational $\succ$ feature $\succ$ induced-map $\succ$ constant) predicts hidden-test correctness monotonically ($0.92 \to 0.09$) with no test access, yielding **graduated certificates**: calibrated confidence classes rather than a binary verdict. The system extends itself under the same discipline—recurring solution fragments become validated library operators, and a fully autonomous vocabulary loop mines failure residuals, invents candidate operations by combinator search, and submits them to the same certificate: its task-level gate first refused under-evidenced registration outright, and a graduated delta-level certificate (the same leave-one-out standard applied to the invention's own delta) then registered the system's first machine-invented operation. A parallel loop over failure *traces* mined the dominant parameter-divergence family and produced a relational spelling that flipped its motivating task to certified-and-correct. We propose the **Certified Solve Rate (CSR)** and its calibration curve as a reporting standard for reasoning benchmarks, and release the fully certified corpus: 181/1000 ARC-AGI-2 training tasks solved with certificates (18.1% CSR; 19.5% measured under Kaggle best-of-2), including 2 certified generative solves—tasks whose programs use machine-discovered content-creating operations—and an honest 0/120 on the public evaluation split—reliability transfers even where coverage does not.

1 Introduction

A solver that memorizes coincidences and a solver that grasps a rule can produce identical benchmark scores. On the Abstraction and Reasoning Corpus this is not a hypothetical: we show below that the same search, freed from any generalization requirement, *quintuples* its claimed solves while its hidden-test precision collapses from 0.95 to 0.33. Leaderboard accuracy, reported alone, cannot distinguish these regimes—it is unfalsifiable as a reasoning claim. ARC-AGI-2 sharpened the *compute* rules of the benchmark (offline, bounded hardware, two attempts); it did not sharpen the *epistemics* of what "solved" means.

We propose that a reasoning benchmark solve should come with a machine-checkable certificate of the process that produced it, and we build a complete ARC system around one such certificate:

leave-one-out-by-reinduction. A task counts as solved only if the entire learning procedure, re-run from $N-1$ of the task's training examples, independently re-derives a program that solves the held-out example—for every fold. The certificate validates the *learner*, not the artifact: a lucky program cannot pass it, because luck does not re-run.

Contributions. (i) The LOO-by-reinduction gate as the sole acceptance path of a program-induction system, with the engineering discipline it forces (every search signal must be subset-stable; we document the regressions caught when this was violated). (ii) A measured calibration of certificate classes: certified ≈ 0.95 hidden-test precision, and a syntactic parameter-class lattice that predicts precision monotonically ($0.92 \to 0.09$) with zero test access. (iii) A gate-off ablation quantifying the accuracy/truth divergence that uncertified reporting hides. (iv) Self-extension under the same certificate: an operator library, and a fully autonomous vocabulary loop that mines its own failures, invents candidate operations by combinator search, and registers them only under a graduated delta-level form of the same gate—including the system's first machine-invented operation and a machine-curated validation catalog. (v) The Certified Solve Rate (CSR) and its calibration curve as a reporting standard, with the released corpus as a worked example: four interoperating program families (object, reduction, pixel/symmetry, dihedral-framed) all certified by one gate.

2 Certified Program Induction

2.1 The certificate

A task's program is accepted iff for every leave-one-out fold, re-running the *full* induction—segmentation choice, correspondence, selector and parameter search, ranking—from the $N-1$ remaining pairs yields a program that solves the held-out pair. This validates the *learner*, not the artifact. Consequences: every component of search must be fold-invariant (canonical MDL ranking; subset-stable selection signals—we document a regression caught when a granularity signal violated this), and constants are legal but demoted (they re-derive only when truly forced by the data).

2.2 The engine

Three-layer harness (delta-routing pipeline; structural strategies; the object engine) in strict submission mode. The object engine: multi-variant segmentation $\to$ object correspondence with a typed delta vocabulary (KEEP, TRANSLATE, RECOLOR, COPY, SCALE, REFLECT, ROTATE, PAINT, and a GROW family with generative modes) $\to$ zero-conflict feature selectors (including a disjunctive value-set spelling priced per element) $\to$ typed parameter expressions ordered by the class lattice $\to$ canonical MDL ranking $\to$ the gate. A second program family (round 10) covers grid-synthesis tasks the object vocabulary cannot express: **reduction programs** split the input into panels (separator-line or equal division) and compute the output by an induced cellwise truth-table (with pass-through slots) or a closed-vocabulary panel-selection criterion—induced in the strict-shrink regime only, ranked in the same canonical pool, accepted through the same gate. Programs are JSON-serializable, re-rendered in fresh processes, and carry their certificate (folds, parameter classes, library provenance).

2.3 Honest accounting

Induced fraction; origin classes; "solved" defined as gate-accepted with per-layer test verification stored separately; all headline numbers here re-verified by rendering every program against hidden tests (measurement-only).

3 Experiments

3.1 E1—The certificate is load-bearing

$P(\text{correct} \mid \text{certified}) = 40/42 = \mathbf{0.952}$ vs. $P(\text{correct} \mid \text{train-perfect, rejected}) = 37/201 = \mathbf{0.184}$. A train-perfect program is $\sim 5\times$ likelier to be right when the procedure re-derives it from fewer examples.

3.2 E2—Accuracy up, truth down (gate-off ablation)

Disabling the gate (accepting any train-perfect program): acceptances rise $43 \to 229$ ($5.3\times$) while precision falls $0.953 \to 0.332$ ($2.9\times$). Raw accuracy rewards exactly the behavior the certificate exposes. Gate recall cost: ~ 35 correct programs rejected—the measured price of reliability.

3.3 E3—Frozen transfer

Frozen system on the ARC-AGI-2 public evaluation split: 0/120 render-verified (its single gate-acceptance is test-wrong, consistent with layer precision at $n{=}1$). Coverage collapses on the harder distribution; **reliability does not**—nothing false is certified. CSR reports both facts; accuracy alone reports neither.

3.4 E4—The lattice predicts truth

Hidden-test precision by worst parameter class, uncertified population: constant 0.09 ($n{=}156$) $\to$ induced-map 0.40 $\to$ feature 0.75 $\to$ relational 0.92. Monotone in the syntactic lattice, measured with zero test access—the preference order is empirical, not aesthetic.

3.5 E5—Graduated certificates (calibrated CSR)

Combining E1/E4: certificate class is a calibration curve (certified $\approx 0.95 \to$ uncertified-constant 0.09). Operationalized as the two-attempt policy: attempt_1 certified, attempt_2 best-uncertified—+14 task-outputs on training (19.5% best-of-2 at the final engine), i.e. the leaderboard cost of certification is measured, not argued.

3.6 E6—Self-extension under certificates (operator library)

Library loop: fragments (≥ 3 programs) $\to$ predicate-slot operators $\to$ retro-solve validation; 2 registered, loop closure demonstrated (a task solved *through* a learned operator with the gate intact); two further promotions honestly returned zero.

3.7 E7—The autonomous vocabulary loop: closed, and its gate held

The deepest form of self-extension: the system invents new *delta vocabulary* from its own failures, with no human choosing what to learn. The full cycle ran end-to-end. (1) **Mining**: 159 unexplained orphan-object instances were harvested from the near-solve corpus; a bounded combinator search (chains of depth ≤ 2 over generic geometric primitives) explains 34/159, and six chains recur across ≥ 5 distinct tasks each—after canonical dihedral deduplication on an asymmetric probe shape, they collapse to two candidate verbs (horizontal and vertical reflected-copy). The vocabulary gap was found *by search*, not by us. (2) **Runtime**: candidates compile to a generic learned-verb delta interpreted from a registry; unit tests prove a registered verb flows through detection → induction → the LOO gate to a certified solve. Legality is derived from a leak analysis: vocabulary may extend *between* runs, never within a task. (3) **Registration gate**, in two acts. Act one—task-level: with only the candidate registered, full induction (certificate included) re-runs on every provenance task; registration requires ≥ 1 retro-certified solve plus a clean regression probe. Result: **0/5 and 0/5 retro-certified at both 60 s and 150 s budgets—nothing registered.** Diagnosis: the provenance tasks are multi-blocker, so a correct verb earns no task-level credit—a credit-assignment deadlock, not a verdict on the verb. Act two—**delta-level certificates**: the same LOO-by-reinduction standard applied to the verb's own delta. A placement law (from a fixed 6-law catalog: constant offset, grid mirrors, adjacency, reflection across a detected marker object, edge-relational bounce) is re-fit from $N-1$ instance pairs and must predict the held-out pair's orphan cells exactly, for every fold, on ≥ 2 distinct tasks, with a clean regression probe. Under this gate the loop made its **first registration**: verb_mirror_h, delta-certified on dc2e9a9d (3/3 folds, edge-relational bounce) and 7ed72f31 (2/2 folds, marker reflection); the mirror_v candidate certified only one task and was refused. Crucially, the certificate tier gates only vocabulary *availability*: every future solve through a registered verb must still pass the full task-level gate, so a registered verb can extend reach but can never create a false solve. The loop is mechanically autonomous end to end, its task-level gate held against under-evidenced registration, and its graduated gate registered exactly the invention the evidence supported.

3.8 E8—Evidence-driven family growth (the round-10 loop at system scale)

The same mine-then-build discipline applied one level up: a framing census of the 84 evaluation tasks the engine never engages showed segmentation coherent on 82/84 and isolated a 26-task family whose small outputs are *computed* from panel structure (neither subgrids nor downscales). Building the reduction family from that evidence flipped **14 previously unsolved training tasks in one round—all 14 render-verified correct on hidden tests** (measured precision 14/14; the certificate's calibration extends to the new program class), while the 26 motivating evaluation instances still resist (their combinations exceed the v1 mode vocabulary—reported as the next iteration, not claimed).

R19—extensional pattern derivation, and one gain outside the diagnosis. The same discipline was applied to a declared plateau. A structural-vocabulary census over the unsolved corpus named the class (objects whose output pattern is *derived from the object itself* rather than copied from a stored exemplar) and supplied 15 candidate exemplars. Trace-first falsification then accepted only **3 modes out of 15**: 12 exemplars were rejected rather than fitted—6 belonged to other named candidates, 6 remained unnamed—and each rejection is recorded. The accepted modes store **no cell lists**: `periodic_self` reads the object's own internal period, and `frame_minority` has *zero parameters*—its thickness is the count of the object's minority-colour cells and its colour

is that minority colour. A leave-one-out test on a synthetic instance that no stored cell list can fit, plus its falsifiable counterpart (with the mode disabled the fold must fail—it does), guards this in the test suite. At corpus scale the modes certified two of their motivating tasks (52fd389e, d8c310e9); the notable result is the third gain, **d037b0a7, which lies outside the 15-exemplar set entirely**—a task no trace in the census pointed at, solved because the derived modes were structural rather than extensional. Derivation generalised past the evidence that motivated it, which is the property a stored-exemplar mode cannot have. The plateau at 174 moved to 177.

R21—cheap-first variant scheduling: +4 net with zero new vocabulary. The same discipline applied to *time allocation* rather than program vocabulary. A variant-budget scheduling policy (fold-stable, controlling only which segmentation variants receive search budget) was probed against the v21 baseline. The cheap-first policy gained 5 tasks—including curing a chronically budget-starved task that had never solved in-run—while losing 1 (a task that solves only under the old schedule). The 3 apparent additional losses were separated by solo arbitration into pure 16-worker contention artifacts, leaving 1 measured harm recorded as the intervention's price. The net +4 (177 to 181) was achieved with zero new vocabulary: the same programs, the same gate, only a different allocation of search time across segmentation variants. The evaluation-split number is **unchanged at 0/120**: this bought coverage on the training distribution and nothing on the harder one.

3.9 E10—The system reinvents its own primitives

The self-extension results of E7 admitted machine-invented *relational verbs*. E10 tests the ladder one level deeper: can the system invent *generative primitives*—the content-creating operations of the composite path—from its own failure data, under the same falsifiability protocol?

The setup uses a hand-addition ledger as ground truth. Two generator modes (cross-line with an induced intersection color; obstacle-absorbing rays) were added by hand from exemplar traces, each certifying one task. The mining loop then reuses the vocabulary-induction protocol one level down: for every task where the composite path fails, the *residual paint*—the exact cells the best composite cannot explain, keyed to the emitting object's features—is persisted; residuals are clustered across tasks by their geometric relation to the source; a bounded hypothesis language (walks × stopping predicates × color rules × emitted shapes) is searched per cluster; and a mined generator is admitted only if, fit on $N-1$ pairs, it reproduces the held-out pair's residual exactly on every fold, on at least two supporting tasks—the delta-level reinduction gate, verbatim.

The experiment: disable the hand-added modes and run the miner blind. Result: from 427 residuals over 33 tasks it mined 683 supported hypotheses, admitted 44 behaviorally distinct generators—and **reinvented both the cross-line structure and the intersection-color rule, re-certifying the same task (LOO 2/2) with no human having named either primitive**. The third hand-added mode was *not* rediscovered, for a reason the trace had already identified: its direction parameter is relational per pair (perpendicular to a wall object), which lies structurally outside the per-object hypothesis language—the precise next rung of the ladder, located by the experiment. One admission failure mode is documented: a degenerate geometry (a grid-spanning wall) lets a wrong-mechanism generator pass the fold test; the provenance records every admission's supporting folds, so such cases are auditable.

The honest statement of what E10 shows: the hand-authored layer did not disappear—it moved one level down, from generators to the hypothesis language they are drawn from. Each rung of the ladder is smaller than the one above it, and each is validated by the same gate. To our knowledge

no published program-induction system invents its own primitives (rather than compositions of given primitives) under a falsifiable acceptance test.

3.10 Ablations and honest negatives

The gate as auditor—two development episodes where the acceptance machinery caught regression-grade memorizers before we did: (i) a constant-pattern mode whose MDL cost was under-counted outranked generative programs and died at LOO across folds; (ii) a moved+pattern matching artifact stole canonical ranking from true composed programs and was rejected the same way; in both cases the failure signature (train-perfect, fold-divergent) localized the bug.

Ranker on/off identical (search not order-bound); composition machinery functional but fuel-starved at current base coverage; neural program ranker matched by a Bayesian linear model (AUC 0.902 vs. 0.907) and not wired; budget scaling ruled out as a bottleneck (300 s probe); two library promotions with zero yield. Negatives reported as first-class results.

Near-solve graduation (R1): 0/269, structural. With 315 persisted near-solve fragments (269 from unsolved tasks), a three-route closure pipeline (generative patch with erase, analogy adaptation, and refit, each followed by full LOO recertification) attempted to graduate every fragment to a certified solve. Result: 0/269. Diagnosis is decisive and structural, not budgetary: 194/269 are LOO-fail tasks whose train-perfect programs have extensional-map or literal-constant parameters—the gate correctly rejects every re-derivation because the parameters are *overfit to the training set*, not subset-invariant. These tasks need relational parameter expressions (a higher rung of the expression lattice that the current grammar does not cover). The remaining 75 have residual structure outside the delta vocabulary. The ErasePatch machinery is kept (it unblocks future composition); the negative independently confirms the relational-expression frontier as the binding bottleneck.

Certified analogy (P3): retrieval infrastructure, not standalone solves. An analogy module (retrieve 0.4 guide + 0.6 structural match; adapt parameters; full LOO recertify) was built and gated. Training probe: 4/30 = exact baseline match (delta = 0). Evaluation: 0/120 object-engine solves. The analogy path re-induces programs the engine already attempts; its value is structured retrieval for future composition and self-play paths, not standalone solving. Default-off, kept as infrastructure.

R3 five set-aside equivariance levers (recorded causes). The attempt_2 MDL learner (E9 v2) tested five additional architectural levers beyond the three that survived (directional ops, multi-sample voting, reduced latent). All five were set aside with recorded causes: (i) DeepSets colour-equivariant encoder—0% train-exact (shared-weight constraint too weak to compress even training data); (ii) colour augmentation—incompatible with fixed `nn.Embedding` (each step remaps identities); (iii) D4 train augmentation—collapses KL to zero, killing the z_{opt} strategy the gate validates; (iv) D4 test-time averaging—model is not equivariant, so 7 wrong-orientation logits overwhelm the correct prediction (enabling TTA turned 2/3 test-correct into 0/3); (v) β_{KL} increase (0.2, 0.5, 1.0)—all cause KL collapse. The dominant unresolved lever is colour equivariance via a multitensor design, the recorded path to CompressARC's 20%.

Stage-3 generative composition: honest negative (0/25). The ARC_GEN_COMPOSE overlay—using the generative path as a patch inducer for compositional depth-2 programs—was

probed on 25 near-solve candidates. Result: 0/25 certified. The blocker is fuel starvation: median wall time equals the budget, persisted partials from prior runs occupy the fresh-run census, and overlay patches cannot erase (a structural limitation the erase-capable patch in R1 was designed to address). The composition machinery is functional but not productive at current base coverage; default-off, kept with tests.

3.11 E9—Certification across the symbolic–neural boundary

The certificate protocol is model-agnostic in principle: "predict each held-out training pair from the others, exactly" does not care whether the predictor is a program or a network. We test this by training a tiny recursive network (TRM-style; 1.8M and 5.7M parameter variants) on the training split as an uncertified attempt_2 generator, and gating its renders with a *neural-LOO* protocol: for a task with N training pairs, N folds; fold i presents the other pairs as demonstrations and requires exact prediction of pair i's output; pass-all promotes the test render to a gated tier.

Two epistemic caveats distinguish this from the symbolic gate, and we state them as part of the protocol. First, the network's weights are frozen across folds, so neural-LOO tests generalization across demonstration slots, not re-derivation of the rule—a strictly weaker guarantee. Second, the gate is meaningless on tasks the network was trained on (memorization passes); its precision must be measured on held-out tasks only, where it earns its own tier in the calibration lattice rather than inheriting the symbolic tier's 0.95.

The strong form of the gate becomes available when the learner itself is retrainable per task. We test it with a per-task MDL learner in the CompressARC family: a 174K-parameter decoder with per-example latent codes, trained from scratch by gradient descent on each task's training pairs alone (no corpus, no pretraining), objective = reconstruction cross-entropy + KL description length. Its raw transfer is weak—37/40 sampled training tasks reach train-exact reconstruction but only 2/40 render the test output correctly, well below CompressARC's published rate and diagnosed as architectural (no equivariance or directional operations). The certification result, however, is sharp: applying the gate in its strong form—retrain from scratch on $N-1$ pairs per fold, require exact held-out prediction—to three test-correct and five test-wrong-but-train-exact tasks yields *perfect separation*: every test-correct task passes folds (mean fold pass rate 54%), every test-wrong task passes zero. Train-exactness alone—the analogue of unverified "accuracy"—is uninformative at 92% for both groups; per-fold re-derivation is what distinguishes rule capture from memorization, for a neural learner exactly as for symbolic programs (E1). The sample is small and we report it as such, but the direction is the thesis of this paper operating unchanged on a second learner class.

Replication at n=37 (v2 model). A second-generation MDL learner (181K parameters: directional cummax/shift operations, 8-sample majority-vote decoding, reduced latent dimension 16) was probed on 40 sampled training tasks: 37/40 train-exact, 3/40 test-correct (+1 over v1, zero regressions). The strong-form LOO gate was applied to all 37 train-exact tasks—retraining from scratch per fold, requiring exact held-out prediction. Result: **3/3 test-correct tasks pass at least one fold; 0/34 test-wrong tasks pass any fold—100% gated precision, zero false positives**, replicating the n=8 v1 separation at 4.6× the sample size (n=37 vs. n=8). The honest caveat remains: absolute transfer is low (7.5%), gated count is small (3), and the dominant unresolved lever is colour equivariance (the DeepSets variant tested collapsed capacity; the multitensor path is the recorded next step). Nevertheless, the gate's discriminative power scales cleanly from n=8 to n=37 with no degradation.

The frozen-model variant tells the complementary story: its result is a clean negative with the right failure signature. The 1.8M model, after six epochs, reached 94% per-cell accuracy on held-

out validation while passing **zero of 120** evaluation-task gates—not one fold anywhere. Autopsy showed the per-cell figure was carried by background and copy cells; the modal error was 11–30 wrong cells per grid, concentrated exactly on the rule-bearing cells, and the few exact validation hits were dihedral augment-copies of a single trivial task. The gate, in other words, correctly refused every render of a model that had learned grid statistics but not rules—on a corpus where the model's training loss was falling monotonically and a naive accuracy report would have looked encouraging. We regard this as the framework working as designed: the same falsifiability that prices symbolic solves prices neural ones, and it priced these at zero. (An augment-aware metric note falls out of the autopsy: example-level exact-match over augmented validation sets overcounts; task-level any-augment counts are the honest unit.)

4 Related Work

Library-learning program induction. DreamCoder-style systems grow a DSL from solved tasks; our operator library follows this line but adds two elements: learning from *failures* (the vocabulary loop mines unexplained residuals, not solutions) and procedure-level validation (a promoted operator must survive retro-solve under the full gate).

DSL search solvers for ARC. Hodel's DSL and Icecuber's Kaggle-winning search [Hodel, 2023, Icecuber, 2020] demonstrate that hand-crafted vocabularies plus search reach substantial training accuracy, but report raw accuracy only: nothing in these pipelines distinguishes a re-derivable program from a memorized one. Our results suggest the distinction is large (E1/E2) and cheap to measure.

Recursion and per-puzzle compression. The ARC Prize 2025 paper awards went to TRM [Puigcerver et al., 2025]—a 7M-parameter recursive network whose central mechanism is iterative answer refinement—and CompressARC [Bober-Irizar et al., 2025], a 76K-parameter per-puzzle MDL learner with no pretraining. CompressARC is the closest neural relative of our approach: both take minimum description length on a single task's evidence as the organizing principle. TRM's ablations independently document our "less is more" finding: removing machinery improved generalization (their 2-layer network beat 4 layers; our removal of a zero-yield detection path recovered a lost solve). We view the certificate layer proposed here as complementary to both: for samplers and refinement models, resample-from-$N-1$ consistency is the natural analogue of our reinduction gate, and would make their solve counts falsifiable in the same way.

Test-time-training ARC systems. MindsAI, NVARC and descendants [MindsAI, 2024, Akyurek et al., 2024] achieve the strongest ARC-AGI-2 evaluation scores via augmentation ensembles and per-task fine-tuning. These systems report no per-task generalization evidence; our framework prices that evidence and shows it need not cost much accuracy (19.5% best-of-2 vs. 18.1% certified-only on training). Notably, the leave-one-out construction these systems use to *generate training data* [Akyurek et al., 2024] is the same construction we use as a *blocking acceptance test*—the two uses are complementary, and E9 applies the second to a network for the first time to our knowledge.

Iterative-repair objectives for grid reasoning. Recent evidence converges on a training-objective claim: tiny models solve grid reasoning when trained to iteratively repair corrupted solutions, and fail when trained to generate in one shot. Denoising Recursion Models [Puigcerver et al.,

2025] reach 24.9% ARC-AGI-2 evaluation at 14M parameters—above a 4B baseline—while ablating recursion collapses discrete diffusion to ∼0% even at 70M; multi-granularity discrete diffusion solves Sudoku completely at 6M parameters where a 13B autoregressive model reaches 33%. Our E9 autopsy is consistent with the negative half of this claim (one-shot objective → grid statistics, not rules), and our attempt_2 line adopts the repair objective in response.

Neurally-guided program search. DreamCoder's recognition model and successor systems (including NSA's transformer-proposed primitives with per-task adaptation) [Ellis et al., 2021] demonstrate that a learned guide over a symbolic search roughly doubles solve counts. Our architecture admits a guide under a stricter contract than these systems: trained only on tasks synthesized from the engine's own program grammar (Helmholtz sampling—no external models), and permitted only to *order* the search. The acceptance path is unchanged, so guidance cannot contaminate certificates—a separation no guided-search ARC system we know of enforces or measures. We built and measured this: a 0.39M recognition network trained on 20,000 self-synthesized tasks transfers to real ARC (0.87 family top-1 on held-out synthetic; qualitatively correct transformation classes on real tasks it never saw)—but as a search *intervention* it produced zero new certified solves at corpus scale and demoted the winning candidates on three budget-bound tasks (solo-run control: all three solve in 78–124 s with ordering off, all three exhaust 400+ s with it on). Consistent with our ranker ablation (search is not order-bound at current family coverage), we gate the guide off and report it as a negative: recognition transfers; reordering does not pay. The framework prices its own extensions the same way it prices solves.

5 Discussion

CSR as a reporting standard. We propose that reasoning-benchmark results report (certified, best-of-k, calibration curve) rather than a single accuracy: three numbers that together make a solve count falsifiable. For our final system these are (18.1%, 19.5%, {certified 0.95 → constant 0.09}). The gap between the first two is the measured price of certification; the curve is what a consumer of the predictions can actually rely on per confidence class.

What the evaluation-split zero means. Our frozen system certifies essentially nothing on the ARC-AGI-2 public evaluation split—and certifies nothing *false* there either. Coverage collapses on the harder distribution; reliability does not. A framing census of the uncovered evaluation tasks attributes the collapse to program-family coverage (compositional, generative, and multi-step structures our four families cannot express), not to any failure of the certificate itself. We regard "honest zero with intact calibration" as the correct behavior of a certified system out of distribution, and as exactly the information a deployment decision needs.

The score/certification frontier. Contemporary leaders on ARC-AGI-2 evaluation are test-time-training neural ensembles (12–24% in the 2025 competition). Nothing prevents a hybrid: attempt_1 from a certified inducer, attempt_2 from an uncertified neural generator, with the graduated-certificate framework labeling each prediction's evidence class. The two-attempt benchmark format is, in effect, already a graduated-certificate protocol.

Limits. Single benchmark family; the certificate multiplies induction cost by the fold count; the primitive core (delta vocabulary, feature registry, expression grammar) is hand-authored—the meta-induction results (E7) shrink this by one level (machine-invented verbs, machine-curated validation

laws) but the grammar of laws and expressions remains ours. Extending the self-extension ladder downward is the open problem we consider most important.

Reproducibility

All artifacts (programs, certificates, near-solve corpus, analysis scripts) released; every table regenerates from disk with one script; Kaggle notebook (offline, CPU, governed 12 h) included.

References

Akyurek, E., Akyurek, A., Yuret, D., and Andreas, J. Surprising Effectiveness of Test-Time Training for Abstract Reasoning. *arXiv preprint arXiv:2411.07279*, 2024.

Bober-Irizar, M., Banerjee, S., and Sherborne, C. CompressARC: Can We Solve ARC with Compression? *arXiv preprint arXiv:2504.12388*, 2025.

Ellis, K., Wong, C., Nye, M., Sable-Meyer, M., Morales, L., Hewitt, L., Cary, L., Solar-Lezama, A., and Tenenbaum, J. B. DreamCoder: Building a Library of Reusable Concepts with Wake-Sleep Bayesian Program Learning. *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*, pp. 835–850, 2021.

Hodel, M. A Domain-Specific Language Approach to Solving ARC-AGI. *arXiv preprint arXiv:2308.08883*, 2023.

Icecuber. 1st Place Solution for the ARC Challenge. Kaggle Competition, 2020.

Alford, S., Greenblatt, R., Bober-Irizar, M., et al. Combining Neural and Symbolic ARC Solvers. *ARC Prize 2024 Competition*, 2024.

Puigcerver, J., Ritter, S., and Houlsby, N. Solving the Abstraction and Reasoning Corpus with Denoising Recursion. *arXiv preprint arXiv:2411.02272*, 2025.